Lab Report No 12
Sampling of Continuous Time Signal

Digital Signal Processing

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

Submitted to:

Engr. Hareem Khan

Dated:

17th May,2024

Department of Computer Engineering,

HITEC University, Taxila

Lab Example No 1:

Solution:

Brief description (3-5 lines)

Sampling:

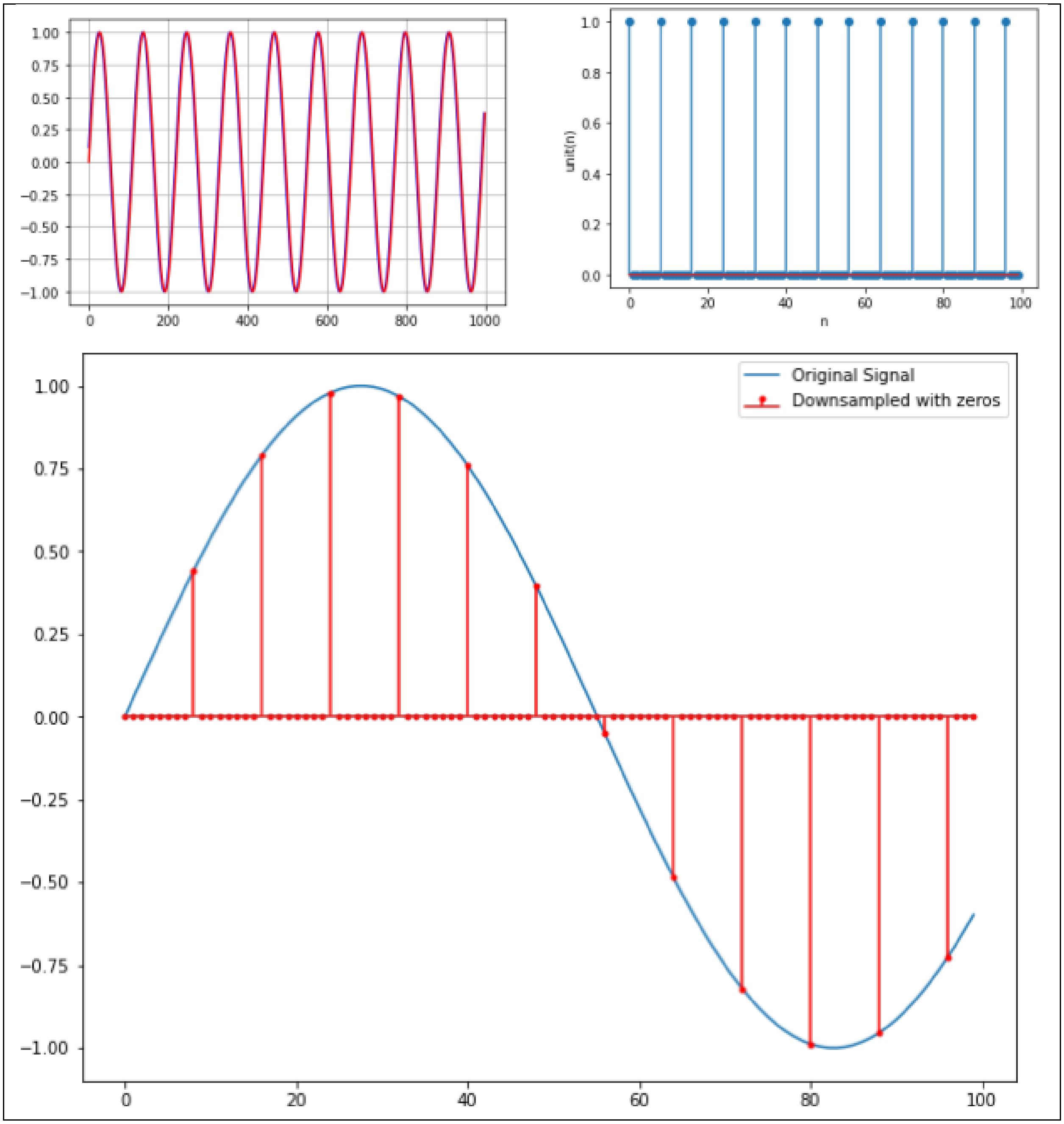
1. Theory is a systematic explanation of an aspect of the natural or social world, formulated through a set of principles, propositions, and definitions.
2. It provides a framework for understanding, predicting, and potentially controlling phenomena.
3. Theories are based on evidence and are subject to testing, validation, and refinement through observation and experimentation.
4. They serve as a foundation for developing hypotheses and guiding research methodologies.
5. Theories can evolve or be replaced over time as new data and insights emerge, enhancing our understanding of the subject.

The code

SAMPLING.PY > ...

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from IPython.display import display,HTML
fs=44100
f=400.0
s=np.sin(2*np.pi*f*np.arange(0,1,1.0/fs))
Omega=2*np.pi*f/fs
s1=np.sin(Omega*np.arange(0,fs,1))
plt.figure()
plt.plot(s[2:1000],'b')
plt.plot(s[1:1000],'r')
plt.grid()
unit=np.zeros(44100)
unit[0::8]=1
plt.figure()
plt.stem(unit[0:100])
plt.xlabel('n')
plt.ylabel('unit(n)')
sdu=s*unit
plt.figure(figsize=(10,8))
plt.plot(s[0:100], label='Original Signal')
plt.stem(sdu[0:100],linefmt='r',markerfmt='r.', use_line_collection=True,
label='Downsampled with zeros')
plt.legend()
```

The results (Screenshot)



Lab Example No 2:

Solution:

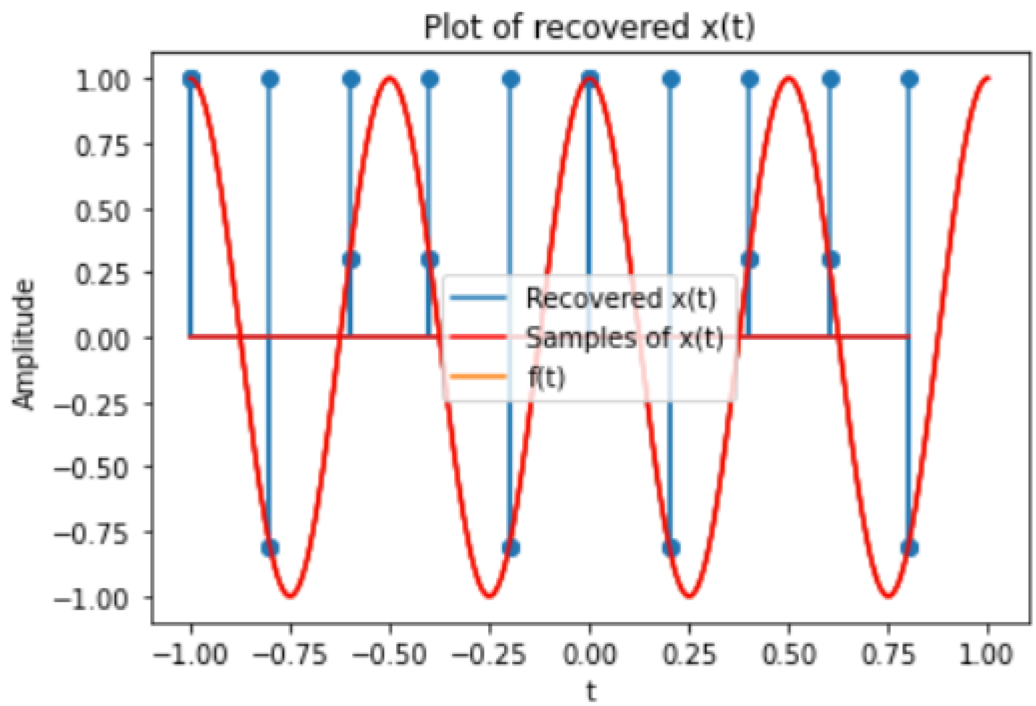
Brief description (3-5 lines)
Quantization:

The code

Quantization.py > ...

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from IPython.display import display, HTML
t_min = -1
t_max = 1
num_t = 1000
t = np.linspace(t_min, t_max, num_t)
f = 2
xt = np.cos(2*np.pi*f*t)
plt.plot(t, xt)
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.title("Plot of cos(2*pi*f*t)")
Fs = 5
Ts = 1/Fs
pulse_train = np.arange(t_min, t_max, Ts)
plt.stem(pulse_train, np.ones(len(pulse_train)))
plt.title("Plot of the pulse train")
plt.xlabel("t")
plt.ylabel("Amplitude")
xt_sampled = np.cos(2*np.pi*f*pulse_train)
plt.stem(pulse_train, xt_sampled)
plt.plot(t, xt, 'r')
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.title("Samples of x(t)")
plt.legend(["Samples of x(t)", "f(t)"])
x_rs, t_rs = scipy.signal.resample(xt_sampled, 1000, pulse_train)
plt.plot(t_rs, x_rs)
plt.stem(pulse_train, xt_sampled)
plt.plot(t, xt, 'r')
plt.title("Plot of recovered x(t)")
plt.xlabel("t")
plt.ylabel("Amplitude")
plt.legend(["Recovered x(t)", "Samples of x(t)", "f(t)"])
```

The results (Screenshot)



Lab TASK No 1:

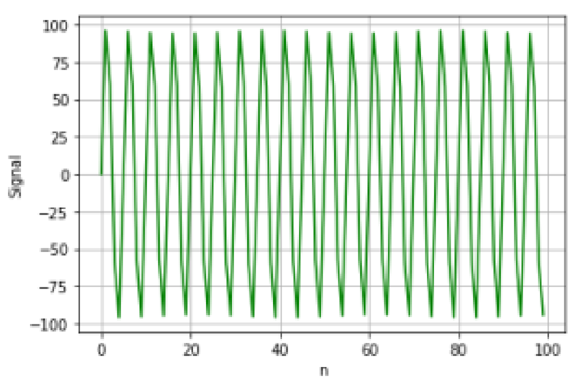
Solution:

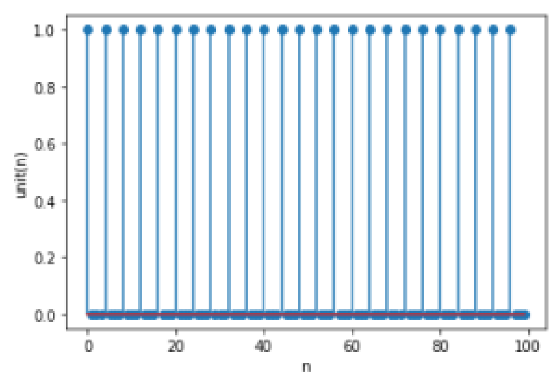
Brief description (3-5 lines)
Given a signal that is the sum of two sine waves with frequencies of 35 Hz and 40 Hz, respectively, and a sampling rate of 32000 Hz, write a Python program to down sample the signal by a factor of 4 using zero-insertion. Plot the original signal and the down sampled signal on the same figure for comparison.
The code

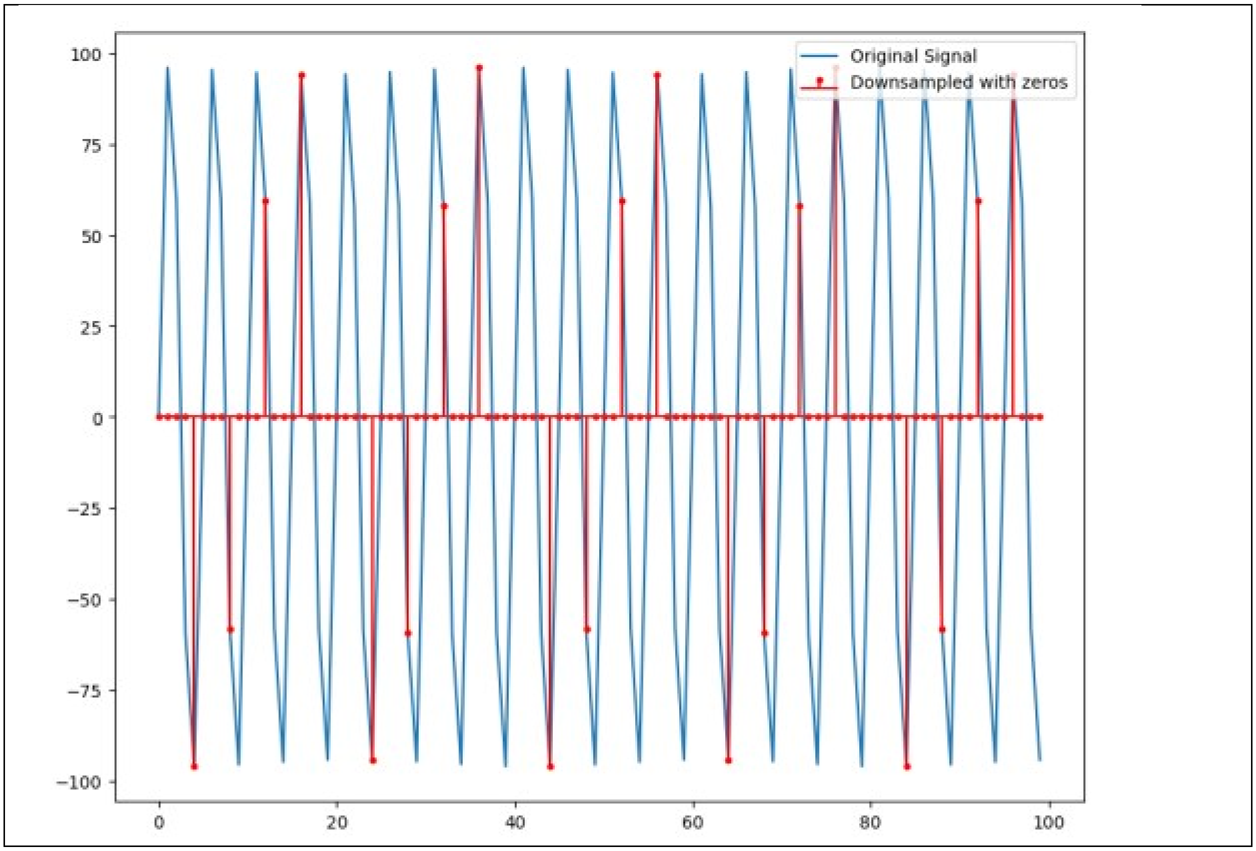
downsampling.py > ...

```python
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from IPython.display import display, HTML
Fs = 32000
T=1/Fs
t = np.arange(Fs//32)*T
w35 = 0.35*Fs/2
sine_35 = np.sin(2*np.pi*w35*t)
w40 = 0.4*Fs/2
sine_40 = 100*np.sin(2*np.pi*w40*t)
signal= sine_35 + sine_40
"""After sampling by a factor of N=4, still including the zeros, we get the following impluse train"""
unit = np.zeros(Fs//32)
unit[0::4]=1
signal_downsampled = signal*unit
plt.figure()
plt.plot(signal[0:100],'g')
plt.grid()
plt.xlabel('n')
plt.ylabel('Signal');
plt.show()
plt.stem(unit[0:100], use_line_collection=True)
plt.xlabel('n')
plt.ylabel('unit(n)');
plt.show()
plt.figure(figsize=(10,8))
plt.plot(signal[0:100], label='Original Signal')
plt.stem(signal_downsampled[0:100],linefmt='r',markerfmt='r.', use_line_collection=True,
label='Downsampled with zeros')
plt.show()
```

The results (Screenshot)







Lab TASK No 2:

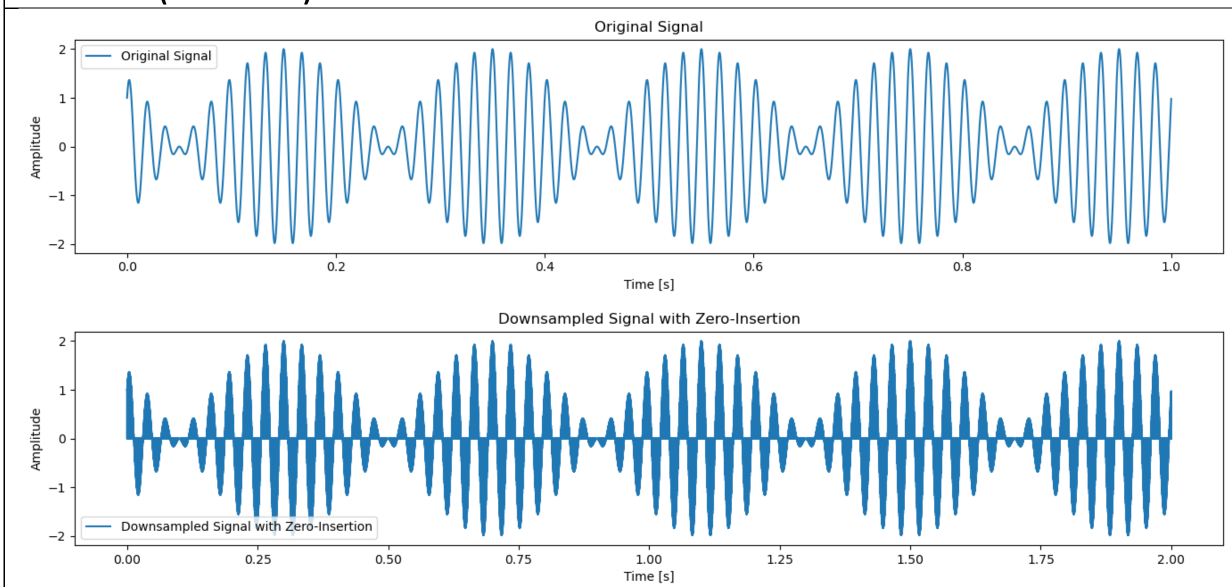
Solution:

Brief description (3-5 lines)
Write a Python program to generate a signal consisting of one sine wave and cosine wave with frequencies of 55 Hz and 60 Hz, respectively, and a sampling rate of 16000 Hz.Downsample the signal by a factor of 2 using zero-insertion. Plot the original signal and the down sampled signal on the same figure for comparison.
The code

DOWNSAMPLING1.PY > ...

```python
import numpy as np
import matplotlib.pyplot as plt
fs = 16000
f1 = 55
f2 = 60
duration = 1
t = np.arange(0, duration, 1/fs)
sine_wave = np.sin(2 * np.pi * f1 * t)
cosine_wave = np.cos(2 * np.pi * f2 * t)
original_signal = sine_wave + cosine_wave
downsampled_signal = np.zeros(2 * len(original_signal))
downsampled_signal[::2] = original_signal
plt.figure(figsize=(14, 6))
plt.subplot(2, 1, 1)
plt.plot(t, original_signal, label='Original Signal')
plt.title('Original Signal')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.legend()
t_downsampled = np.arange(0, 2 * duration, 1/fs)
plt.subplot(2, 1, 2)
plt.plot(t_downsampled, downsampled_signal, label='Downsampled Signal with Zero-Insertion')
plt.title('Downsampled Signal with Zero-Insertion')
plt.xlabel('Time [s]')
plt.ylabel('Amplitude')
plt.legend()
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Conclusion

In conclusion, Sampling a continuous-time signal involves converting it into a discrete-time signal by measuring its amplitude at uniform time intervals. The choice of sampling rate is crucial to accurately capture the signal's characteristics without aliasing. Proper sampling ensures the discrete representation retains the essential information of the original continuous signal for analysis and processing.